TD 3 : SQL

Fonctions d'agrégation, requêtes imbriquées

2026-10-02

Avec solutions

L3 MIASHS
Université Paris Cité
Année 2025
Course Homepage
Moodle

Objectifs

- requêtes imbriquées,
- fonctions d'agrégation.

Prérequis

- schéma `pagila`,
- cours Requêtes imbriquées, CTEs.
- cours Aggrégation, Partitions, Fenêtres.

Les questions qui suivent concernent la base `pagila`. Il est donc nécessaire de changer le `search_path` pour inclure le schéma `pagila`.

Questions

1. Quels sont les films (identifiant et titre) de la catégorie `animation` ? (66 lignes)

Solution

```
SELECT DISTINCT f.film_id, f.title
FROM film f
   JOIN film_category fc ON (f.film_id = fc.film_id)
   JOIN category c ON (fc.category_id = c.category_id)
WHERE c.name ilike 'animation';
```

2. Quelles sont les films (identifiant et titre) qui sont disponibles dans deux magasins différents, c'est-à-dire présents dans deux inventaires différents ? Proposer **deux versions**, une première avec une requête imbriquée et une seconde sans sous-requête. (563 lignes)

Solution

Version avec `IN` et une sous-requête :

```
SELECT DISTINCT f1.film_id, f1.title
FROM film AS f1
   JOIN inventory AS i1 ON (f1.film_id = i1.film_id)
WHERE f1.film_id IN (
   SELECT f2.film_id
   FROM film AS f2 JOIN inventory AS i2 ON (f2.film_id = i2.film_id)
   WHERE i2.store_id != i1.store_id
)
ORDER BY f1.film_id;
```

Solution

Version sans sous-requête :

```
SELECT DISTINCT f.film_id, f.title
FROM film AS f
   JOIN inventory AS i1 ON (f.film_id = i1.film_id)
   JOIN inventory AS i2 ON (f.film_id = i2.film_id)
WHERE i2.store_id != i1.store_id
ORDER BY f.film_id;
```

3. Donner le nombre de films dans la base ? (1000)

Solution

```
SELECT COUNT(film_id)
FROM film;
```

4. Donner le nombre d'actrices ou d'acteurs par film (identifiant et titre) ? Attention à conserver les films sans acteurs (il y en a 3). Ordonner la liste par ordre décroissant du nombre d'acteurs. La première ligne est `508 LAMBS CINCINATTI  15`

Solution

```
SELECT f.film_id, f.title, COUNT(fa.actor_id) AS nb_act
FROM film f
   LEFT JOIN film_actor fa USING (film_id)
GROUP BY f.film_id, f.title
ORDER BY nb_act DESC;
```

5. Donner la liste des films (identifiant et titre) où le nombre d'actrices ou d'acteurs est au moins de 10 ? (48 lignes) Ordonner la liste par ordre décroissant du nombre d'acteurs.

Solution

```
SELECT f.film_id, f.title, COUNT(fa.actor_id) AS nb_act
FROM film f
   JOIN film_actor fa USING (film_id)
GROUP BY f.film_id, f.title
HAVING COUNT(fa.actor_id) >= 10
ORDER BY nb_act DESC;
```

6. Donner les nombres moyen, minimum et maximum d'actrices ou d'acteurs par film ? Attention à prendre en compte les films sans acteurs. Arrondir la moyenne à 1 chiffre après la virgule. (Résultat : 5.5, 0, 15)

Solution

```
WITH nb_act_film AS (
   SELECT f.film_id, COUNT(fa.actor_id) AS nb_act
   FROM film f
      LEFT JOIN film_actor fa USING (film_id)
   GROUP BY f.film_id
)
SELECT ROUND(AVG(nb_act),1) AS nb_act_avg, MIN(nb_act) AS nb_act_min, MAX(nb_act) AS nb_act_ma
FROM nb_act_film;
```

7. Donner trois actrices ou acteurs (identifiant, nom et prénom) qui ont joué dans le plus grand nombre de films ? (identifiants 107, 102, 198 et il y a unicité du résultat)

Solution

```
SELECT a.actor_id, a.last_name, a.first_name, COUNT(film_id) AS nb_film
FROM actor a LEFT JOIN film_actor fa USING(actor_id)
GROUP BY a.actor_id, a.last_name, a.first_name
ORDER BY nb_film DESC
LIMIT 3;
```

8. Donner la liste des actrices/acteurs (identifiant, nom et prénom) qui ont joué dans au plus 20 films. (19 lignes) Ordonner la liste par ordre décroissant du nombre de films.

Solution

```
SELECT a.actor_id, a.last_name, a.first_name, COUNT(fa.film_id) AS nb_film
FROM actor a LEFT JOIN film_actor fa USING(actor_id)
GROUP BY a.actor_id, a.last_name, a.first_name
HAVING COUNT(fa.film_id) <= 20
ORDER BY nb_film DESC;
```

9. Ecrire une requête qui calcule pour chaque film, le nombre de locations et le nombre d'actrices ou d'acteurs. Comment rechercher une éventuelle corrélation entre ces deux variables ? (Pour le film d'identifiant 1, le nombre de locations est 23 et le nombre d'acteurs est 10). Attention à ne pas oublier les films sans location et les films sans acteurs.

Solution

```
WITH nb_act_film AS (
   SELECT film_id, COUNT(actor_id) AS nb_act
   FROM film
      LEFT JOIN film_actor USING(film_id)
   GROUP BY film_id
),
nb_loc_film AS (
   SELECT film_id, COUNT(rental_id) AS nb_loc
   FROM film
      LEFT JOIN inventory USING(film_id)
      LEFT JOIN rental USING(inventory_id)
   GROUP BY film_id
)
SELECT f.film_id, f.title, naf.nb_act, nlf.nb_loc
FROM film f
   JOIN nb_act_film naf USING(film_id)
   JOIN nb_loc_film nlf USING(film_id)
ORDER BY naf.nb_act, nlf.nb_loc;
```

10. Quels sont les actrices ou acteurs (identifiant, nom et prénom) qui ont joué plus que la moyenne des actrices/acteurs, ordonnés par le nombre décroissant de films ? (94 lignes). Proposer **deux versions**, une première avec une CTE et une requête imbriquée et une seconde, avec des CTEs et une jointure. Pourquoi la seconde version est-elle plus performante ?

Solution

Avec une CTE et une sous-requête :

```
WITH nb_film_act AS (
   SELECT a.actor_id, COUNT(fa.film_id) AS nb_film
   FROM actor a LEFT JOIN film_actor fa USING(actor_id)
   GROUP BY a.actor_id
)
SELECT a.actor_id, a.last_name, a.first_name, nfa.nb_film
FROM actor a
   JOIN nb_film_act nfa
   USING(actor_id)
WHERE nfa.nb_film > (
   SELECT AVG(nb_film)
   FROM nb_film_act
)
ORDER BY nfa.nb_film DESC;
```

Solution

Avec deux CTEs et une jointure :

```
WITH nb_film_act AS (
   SELECT a.actor_id, COUNT(fa.film_id) AS nb_film
   FROM actor a LEFT JOIN film_actor fa USING(actor_id)
   GROUP BY a.actor_id
),
avg_film_act AS (
   SELECT AVG(nb_film) AS avg_film
   FROM nb_film_act
)
SELECT a.actor_id, a.last_name, a.first_name, nfa.nb_film
FROM actor a
   JOIN nb_film_act nfa USING(actor_id)
   CROSS JOIN avg_film_act afa
WHERE nfa.nb_film > afa.avg_film
ORDER BY nfa.nb_film DESC;
```

11. Quels sont les acteurs (identifiant, nom et prénom) qui ont joué dans toutes les catégories de film ? (11 lignes) Proposer une solution utilisant les fonctions d'agrégation puis, si possible, une solution sans fonction d'agrégation.

Solution

Solution avec fonction d'agrégation :

```
WITH nb_cat AS (
   SELECT count(category_id) nb
   FROM category
)
SELECT a.actor_id, a.last_name, a.first_name
FROM actor a
   LEFT JOIN film_actor fa USING(actor_id)
   LEFT JOIN film f USING(film_id)
   LEFT JOIN film_category fc USING(film_id)
GROUP BY a.actor_id
HAVING COUNT(DISTINCT fc.category_id) =
   (SELECT * FROM nb_cat)
ORDER BY a.actor_id;
```

La réponse à cette requête en algèbre relationnelle se fait avec une *division* ou $\div$. La division est une opération qui permet de trouver les éléments d'une table qui sont associés à tous les éléments d'une autre table.
En algèbre relationnelle, la requête s'écrit ainsi :

$$\pi_{\texttt{actor_id,category_id}}(\texttt{film_actor} \bowtie \texttt{film_category}) \div \pi_{\texttt{category}}(\texttt{film_category})$$

Cette opération n'éxiste pas en SQL, mais on peut la simuler avec des requêtes imbriquées ou des CTEs.

Solution

Solution sans fonction d'agrégation :

```
WITH ActCat AS (
   SELECT actor_id, category_id
   FROM film_actor fa
      JOIN film_category fc ON (fa.film_id=fc.film_id)
),
ActNot AS (
   SELECT DISTINCT actor_id
   FROM actor, category
   WHERE (actor_id,category_id) NOT IN (SELECT * FROM ActCat)
),
ActId AS (
   SELECT actor_id FROM actor
   EXCEPT
   SELECT * FROM ActNot
)
SELECT actor_id,first_name,last_name
FROM actor JOIN ActId USING(actor_id)
ORDER BY actor_id;
```

Solution

La version suivante suit fidèlement le plan d'exécution le plus élémentaire pour réaliser la division.

```
WITH
  ActCat AS (
    SELECT actor_id, category_id
    FROM film_actor fa JOIN film_category fc ON (fa.film_id=fc.film_id)),
  ActCrosCat AS (
    SELECT  actor_id, category_id
    FROM  actor, category),
  ActNotCat AS (
    SELECT *
    FROM ActCrosCat
    EXCEPT
    SELECT *
    FROM ActCat),
  ActId AS (
    SELECT actor_id
    FROM actor
    EXCEPT
    SELECT actor_id
    FROM ActNotCat)
SELECT actor_id, first_name, last_name
FROM actor JOIN ActId USING(actor_id)
ORDER BY actor_id;
```

12. Bonus : existe-t-il des acteurs qui n'ont joué avec aucun autre acteur ? (Réponse : non)

Solution

Avec une jointure externe

```
WITH R1 AS (
   SELECT DISTINCT fa1.actor_id
   FROM film_actor as fa1
      JOIN film_actor as fa2
      ON (fa1.film_id = fa2.film_id AND
         fa1.actor_id != fa2.actor_id)
)
SELECT actor_id
FROM actor LEFT JOIN R1 USING(actor_id)
WHERE actor_id IS NULL;
```

Avec une CTE et une différence

Solution

```
WITH R1 AS (
   SELECT fa1.actor_id
   FROM film_actor as fa1
   JOIN film_actor as fa2
   ON (fa1.film_id = fa2.film_id AND
     fa1.actor_id != fa2.actor_id)
)
(
   SELECT actor_id FROM actor
)
EXCEPT
(
   SELECT actor_id FROM R1
);
```

Avec une requête imbriquée `NOT IN` et une CTE :

Solution

```
WITH R1 AS (
   SELECT fa1.actor_id
   FROM film_actor as fa1
   JOIN film_actor as fa2
   ON (fa1.film_id = fa2.film_id AND
     fa1.actor_id != fa2.actor_id)
)
SELECT actor_id
FROM actor
WHERE actor_id NOT IN (
    SELECT actor_id
    FROM R1
);
```

ou avec `NOT EXISTS`

Solution

```
SELECT actor_id
FROM
  actor a
WHERE
  NOT EXISTS (
    SELECT fa2.actor_id
    FROM
      film_actor fa1
    JOIN
      film_actor fa2
    ON
     (fa1.actor_id=a.actor_id AND
      fa2.actor_id<> a.actor_id AND
      fa1.film_id=fa2.film_id)
);
```

Pour aller plus loin : les vues

Les *vues* permettent de donner un nom à une requête afin de pouvoir l'appeler plus tard sans la réécrire à chaque fois. Une vue s'enregistre dans un schéma. Par exemple, dans le schéma `World`, on pourrait créer une vue `VillesRepublic` qui contiendrait toutes les villes de la table `city` qui sont dans une république.

On crée une vue avec `CREATE VIEW nom AS requete`. Étant donné que vous ne pouvez écrire que dans votre schéma personnel, il faudra nommer vos vues `entid.nom` où `entid` est votre identifiant en salle de TP.

```
CREATE VIEW entid.VillesRepublic AS
   SELECT
      ci.*
   FROM
      world.country as co JOIN
      world.city as ci USING (countrycode)
   WHERE
      co.governmentform like '%Republic%';
```

Cette requête crée une vue dans votre schéma personnel. Désormais, si on veut sélectionner les villes qui sont dans une république et dont la population est supérieure à 1000000, on pourra simplement écrire :

```
SELECT *
FROM
    entid.VillesRepublic
WHERE
    population>=1000000;
```

> Remarquez la différence entre `WITH` et une vue. `WITH` nomme une requête temporairement, seulement à l'échelle de la requête courante tandis qu'une vue est enregistrée de façon permanente. Cependant, chaque fois que vous appelez votre vue, elle est réévaluée par le système de gestion de base de données.

Notez aussi que SQL n'est pas sensible à la casse. La vue `entid.VillesRepublic` peut être aussi désignée par `entid.villesrepublic`.

Pour supprimer une vue existante, on utilise la commande `DROP VIEW` suivie du nom de la vue à supprimer. Par exemple l'instruction

```
DROP VIEW entid.VillesRepublic ;
```

supprime la vue créée précédemment.

13. **QUESTION** Dans votre schéma personnel `entid` (il faut remplacer `entid` par votre nom de login), écrire une vue `film_id_horror` qui renvoie la liste des films de catégorie 'Horror'. Puis avec cette vue, écrire une requête qui affiche les titres des films de cette catégorie. (56 lignes)

Solution

```
CREATE VIEW entid.film_id_horror
AS
(
  SELECT film_id
  FROM film_category JOIN   category USING(category_id)
  WHERE category.name='Horror'
) ;

SELECT film.title
FROM entid.film_id_horror
   JOIN film USING (film_id);
```